

1. Write a menu driven program to perform the following operations of a stack using array by using suitable user defined functions for each case:

- a) Check if the stack is empty
- b) Display the contents of stack
- c) Push
- d) Pop

Source Code:

```
#include <stdio.h>

int size = 5;
int stack[5];
int TOP = -1;

void DISPLAY(){
    if (TOP == -1) printf("Stack is Empty\n");
    else {
        printf("Stack Elements:\n");
        for(int i=TOP; i>=0; i--){
            printf("%d", stack[i]);
            printf("\n");
        }
    }
    return;
}

void PUSH(int value){
    TOP++;
    if (TOP < size){
        stack[TOP] = value;
        DISPLAY();
        return;
    } else {
        printf("OVERFLOW\n");
        TOP--;
        return;
    }
}

void POP(){
    if (TOP == -1){
        printf("UNDERFLOW\n");
        return;
    } else {
        TOP--;
        DISPLAY();
        return;
    }
}

int main(){
    int choice, value;
```

```
printf("1. Check if the stack is empty\n");
printf("2. Display the contents of stack\n");
printf("3. PUSH Element\n");
printf("4. POP Element\n");
printf("5. Exit\n");

while(1){
    printf("Enter Operation Choice: ");
    scanf("%d", &choice);

    switch (choice) {
    case 1:
        if (TOP == -1) printf("Stack is Empty\n");
        else printf("Stack is Not Empty\n");
        break;

    case 2:
        DISPLAY();
        break;

    case 3:
        printf("Enter new element: ");
        scanf("%d", &value);
        PUSH(value);
        break;
    case 4:
        POP();
        break;
    case 5:
        return 0;
    }
}
```

Output:

```
PS D:\DS\LAB5> gcc p1.c -o p1.exe
PS D:\DS\LAB5> .\p1.exe
1. Check if the stack is empty
2. Display the contents of stack
3. PUSH Element
4. POP Element
5. Exit
Enter Operation Choice: 1
Stack is Empty
Enter Operation Choice: 3
Enter new element: 10
Stack Elements:
10
Enter Operation Choice: 3
Enter new element: 20
Stack Elements:
20
10
Enter Operation Choice: 2
Stack Elements:
20
10
Enter Operation Choice: 4
Stack Elements:
10
Enter Operation Choice: 1
Stack is Not Empty
Enter Operation Choice: 5
PS D:\DS\LAB5>
```

2. Write a menu driven program to perform the following operations of a stack using single linked list by using suitable user defined functions for each case:

- a) Check if the stack is empty
- b) Display the contents of stack
- c) Push
- d) Pop

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *TOP = NULL;

void DISPLAY(){
    if (TOP == NULL) printf("Stack is Empty\n");
    else {
        struct node *ptr = TOP;
        printf("Stack Elements:\n");
        while(ptr != NULL){
            printf("%d", ptr->data);
            printf("\n");
            ptr = ptr->next;
        }
        printf("\n");
    }
    return;
}

void PUSH(int value){
    struct node *new;
    new = (struct node*) malloc(sizeof(struct node));
    new->data = value;
    new->next = TOP;
    TOP = new;
    DISPLAY();
    return;
}

void POP(){
    if (TOP == NULL){
        printf("UNDERFLOW\n");
        return;
    }
    struct node *temp;
    temp = TOP;
    TOP = temp->next;
    free(temp);
}
```

```

    DISPLAY();
    return;
}

int main(){
    int choice, value;
    printf("1. Check if the stack is empty\n");
    printf("2. Display the contents of stack\n");
    printf("3. PUSH Element\n");
    printf("4. POP Element\n");
    printf("5. Exit\n");

    while(1){
        printf("Enter Operation Choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (TOP == NULL) printf("Stack is Empty\n");
                else printf("Stack is Not Empty\n");
                break;

            case 2:
                DISPLAY();
                break;

            case 3:
                printf("Enter new element: ");
                scanf("%d", &value);
                PUSH(value);
                break;

            case 4:
                POP();
                break;

            case 5:
                return 0;

        }
    }
}

```

Output:

```
PS D:\DS\LAB5> gcc p2.c -o p2.exe
PS D:\DS\LAB5> .\p2.exe
1. Check if the stack is empty
2. Display the contents of stack
3. PUSH Element
4. POP Element
5. Exit
Enter Operation Choice: 1
Stack is Empty
Enter Operation Choice: 3
Enter new element: 10
Stack Elements:
10

Enter Operation Choice: 3
Enter new element: 20
Stack Elements:
20
10

Enter Operation Choice: 2
Stack Elements:
20
10

Enter Operation Choice: 4
Stack Elements:
10

Enter Operation Choice: 1
Stack is Not Empty
Enter Operation Choice: 5
PS D:\DS\LAB5>
```

3. Write a program in C to convert decimal to binary using stack.

Source Code:

```
#include <stdio.h>

int size = 32;
int stack[32];
int TOP = -1;

void PUSH(int value){
    TOP++;
    if (TOP < size){
        stack[TOP] = value;
        return;
    } else {
        printf("OVERFLOW\n");
        TOP--;
        return;
    }
}

int POP(){
    if (TOP == -1){
        printf("UNDERFLOW\n");
    }
    return stack[TOP--];
}

int main(){
    int num;
    printf("Enter Number: ");
    scanf("%d", &num);

    int r;
    while (num != 0){
        r = num%2;
        num = num/2;
        PUSH(r);
    }

    printf("Binary: ");
    while (TOP != -1){
        printf("%d ", POP());
    }
}
```

Output:

```
PS D:\DS\LAB5> gcc p3.c -o p3.exe
PS D:\DS\LAB5> .\p3.exe
Enter Number: 25
Binary: 1 1 0 0 1

PS D:\DS\LAB5>
```

4. Write a program in C to print the reverse of a string using stack.

Source Code:

```
#include <stdio.h>

int size = 100;
char stack[100];
int TOP = -1;

void PUSH(char value){
    TOP++;
    if (TOP < size){
        stack[TOP] = value;
        return;
    } else {
        printf("OVERFLOW\n");
        TOP--;
        return;
    }
}

char POP(){
    if (TOP == -1){
        printf("UNDERFLOW\n");
    } else if (TOP >= 0){
        return stack[TOP--];
    } return '\0';
}

int main(){
    char str[100];
    printf("Enter String: ");
    scanf("%s", str);

    int i = 0;
    while (str[i] != '\0'){
        PUSH(str[i]);
        i++;
    }
    printf("Reverse String: ");
    while (TOP != -1){
        printf("%c", POP());
    }
    return 0;
}
```

Output:

```
PS D:\DS\LAB5> gcc p4.c -o p4.exe
PS D:\DS\LAB5> .\p4.exe
Enter String: hello
Reverse String: olleh

PS D:\DS\LAB5>
```

5. Write a program in C to check if the brackets are balanced or not, using stack.

Source Code:

```
#include <stdio.h>

int size = 100;
char stack[100];
int TOP = -1;

void PUSH(char value){
    TOP++;
    if (TOP < size){
        stack[TOP] = value;
        return;
    } else {
        printf("OVERFLOW\n");
        TOP--;
        return;
    }
}

char POP(){
    if (TOP == -1){
        printf("UNDERFLOW\n");
    } else if (TOP >= 0){
        return stack[TOP--];
    } return '\0';
}

int main(){
    char str[100];
    printf("Enter String: ");
    scanf("%s", str);

    int i = 0;
    int flag = 1;
    while (str[i] != '\0'){
        if (str[i] == '(' || str[i] == '{' || str[i] == '[')
            PUSH(str[i]);
        else if (str[i] == ')' || str[i] == '}' || str[i] == ']){
            char popped = POP();
            if ((str[i] == ')' && popped != '(') ||
                (str[i] == '}' && popped != '{') ||
                (str[i] == ']' && popped != '[')) {
                flag = 0;
                break;
            }
        }
        i++;
    }

    if (TOP == -1 && flag == 1) printf("Brackets are balanced\n");
    else printf("Brackets are not balanced\n");
}
```

```
    return 0;  
}
```

Output:

```
PS D:\DS\LAB5> gcc p5.c -o p5.exe  
PS D:\DS\LAB5> .\p5.exe  
Enter String: {[()]}  
Brackets are balanced  
  
PS D:\DS\LAB5>
```